

House Price Prediction using Multiple Linear Regression

Model 1: Without Address Feature

Model 2: With Address Feature

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pylab as pl
```

```
data = pd.read_csv("../drive/MyDrive/Colab Notebooks/housePrice.csv")
```

```
data.head(5)
```

	Area	Room	Parking	Warehouse	Elevator	Address	Price	Price(USD)
0	63	1	True	True	True	Shahran	1.850000e+09	61666.67
1	60	1	True	True	True	Shahran	1.850000e+09	61666.67
2	79	2	True	True	True	Pardis	5.500000e+08	18333.33
3	95	2	True	True	True	Shahrake Qods	9.025000e+08	30083.33

Next steps:

[Generate code with data](#)[New interactive sheet](#)

```
data.describe()
```

	Room	Price	Price(USD)
count	3479.000000	3.479000e+03	3.479000e+03
mean	2.079908	5.359023e+09	1.786341e+05
std	0.758275	8.099935e+09	2.699978e+05
min	0.000000	3.600000e+06	1.200000e+02
25%	2.000000	1.418250e+09	4.727500e+04
50%	2.000000	2.900000e+09	9.666667e+04
75%	2.000000	6.000000e+09	2.000000e+05
max	5.000000	9.240000e+10	3.080000e+06

▼ Data Cleaning

```
df = data[data["Area"].notna()]
```

```
df["Area"] = df["Area"].str.replace(",","")
df["Area"] = pd.to_numeric(df["Area"])
```

```
df["Area"].describe()
```

	Area
count	3.479000e+03
mean	8.744000e+06
std	3.167266e+08
min	3.000000e+01
25%	6.900000e+01
50%	9.000000e+01
75%	1.200000e+02
max	1.616000e+10

dtype: float64

```
df = df[df['Address'].notna()]
```

```
df.rename(columns={"Price(USD)": "Price_USD"}, inplace=True)
```

```
df["Parking"] = df["Parking"].astype(int)
df["Warehouse"] = df["Warehouse"].astype(int)
df["Elevator"] = df["Elevator"].astype(int)
```

```
df = pd.get_dummies(df, columns=["Address"], drop_first=True)
```

```
df = df.astype(int)
df = df[df["Area"] < 600]
```

Data Frame is cleaned

df

	Area	Room	Parking	Warehouse	Elevator	Price	Price_USD	Address_A
0	63	1	1	1	1	1850000000	61666	
1	60	1	1	1	1	1850000000	61666	
2	79	2	1	1	1	550000000	18333	
3	95	2	1	1	1	902500000	30083	
4	123	2	1	1	1	7000000000	233333	
...	...	...	...	...	...	...	...	...
3474	86	2	1	1	1	3500000000	116666	
3475	83	2	1	1	1	6800000000	226666	
3476	75	2	0	0	0	365000000	12166	
3477	105	2	1	1	1	5600000000	186666	
3478	82	2	0	1	1	360000000	12000	

3436 rows × 198 columns

✓ Model 1: Without Address Feature

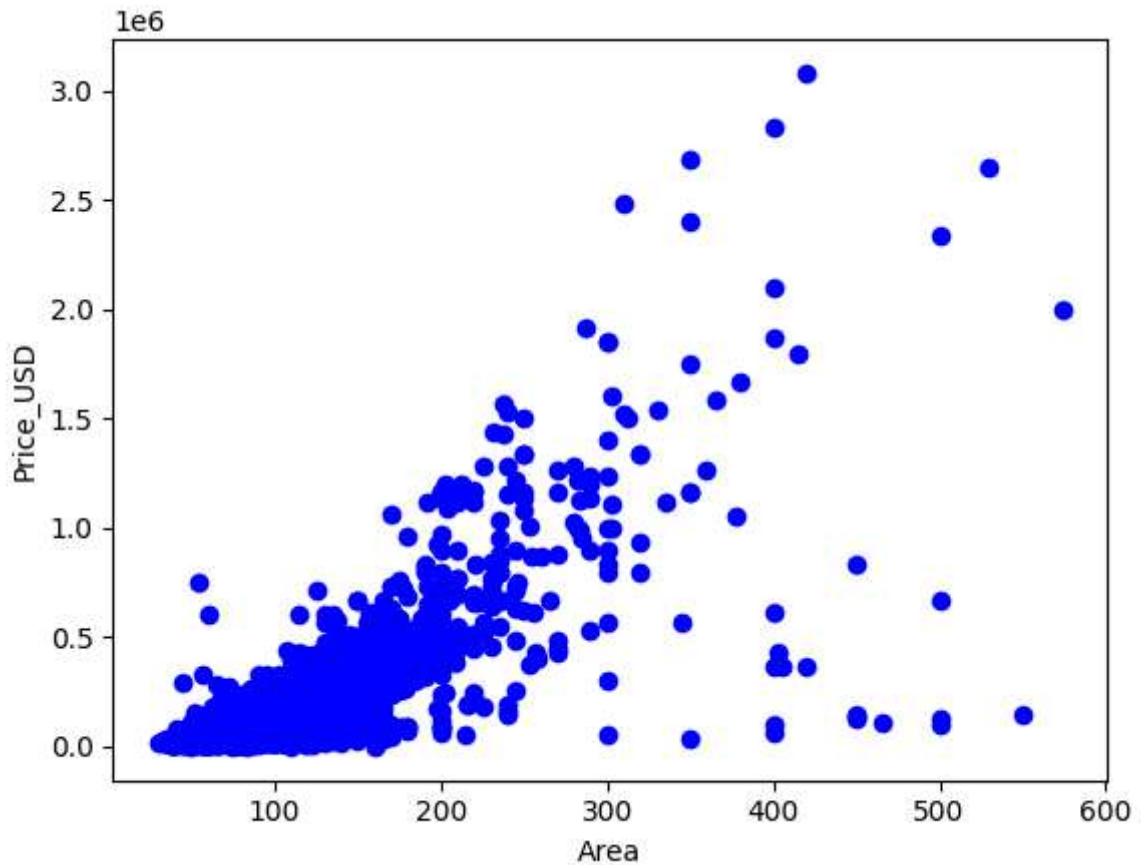
```
cdf = df[["Area", "Room", "Parking", "Warehouse", "Elevator", "Price_USD"]]
cdf.head(5)
```

	Area	Room	Parking	Warehouse	Elevator	Price_USD	
0	63	1	1	1	1	61666	
1	60	1	1	1	1	61666	
2	79	2	1	1	1	18333	
3	95	2	1	1	1	30083	
4	123	2	1	1	1	233333	

Next steps:

[Generate code with cdf](#)[New interactive sheet](#)

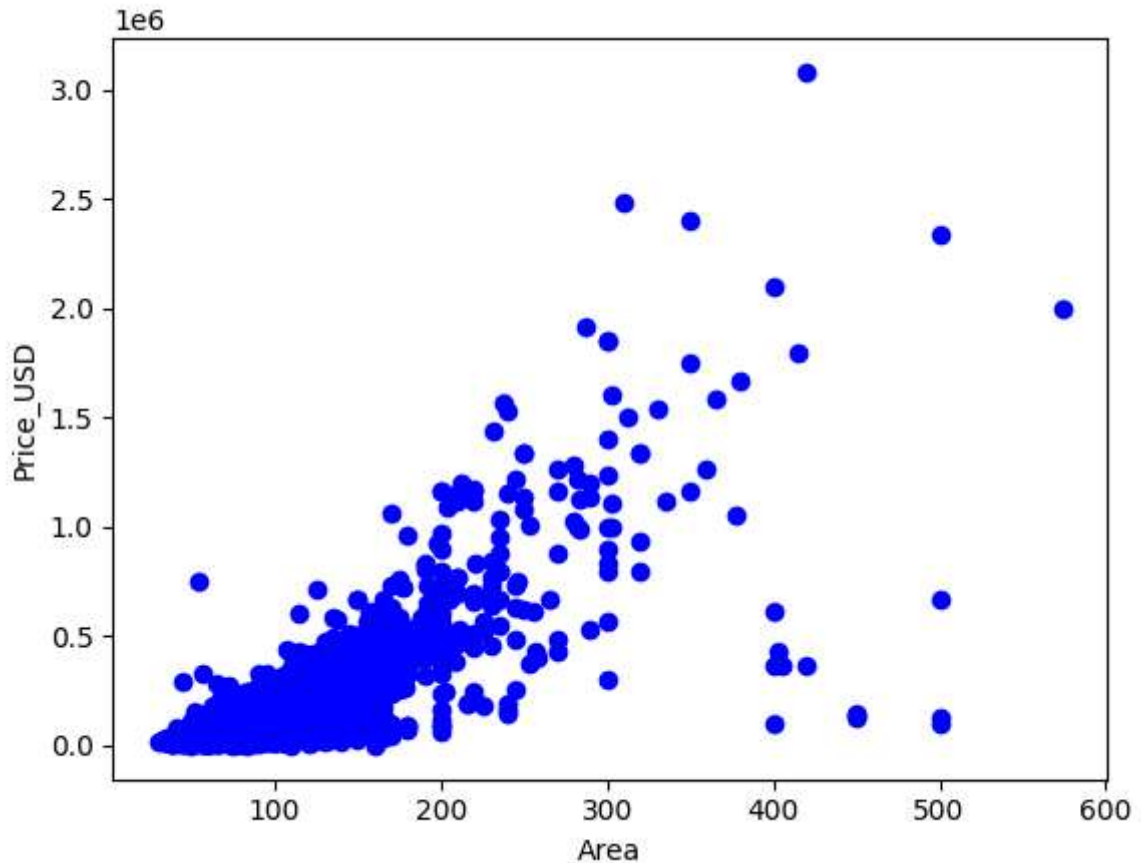
```
plt.scatter(cdf.Area, cdf.Price_USD, color="blue")
plt.xlabel("Area")
plt.ylabel("Price_USD")
plt.show()
```



```
mks = np.random.rand(len(df)) < 0.8
train = cdf[mks]
test= cdf[~mks]
```

```
plt.scatter(train.Area, train.Price_USD, color="blue")
plt.xlabel("Area")
```

```
plt.ylabel("Price_USD")
plt.show()
```



```
from sklearn import linear_model
regr = linear_model.LinearRegression()
x = np.asanyarray(train[["Area", "Room", "Parking", "Warehouse", "Elevator"]])
y = np.asanyarray(train[["Price_USD"]])
regr.fit(x, y)
print("Coefficients: ", regr.coef_)
print("Intercept: ", regr.intercept_)
```

```
Coefficients:  [[ 3480.20295325 -12811.00404761 -7756.72384903  45053.82736738
 31243.58034994]]
Intercept:  [-222061.01627886]
```

```
y_hat = regr.predict(test[["Area", "Room", "Parking", "Warehouse", "Elevator"]])
x = np.asanyarray(test[["Area", "Room", "Parking", "Warehouse", "Elevator"]])
y = np.asanyarray(test[["Price_USD"]])
print("Residual sum of squares: %.2f" % np.mean((y_hat - y) ** 2))
print("Variance score: %.2f" % regr.score(x, y))
```

```
Residual sum of squares: 37592047306.32
Variance score: 0.54
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2732: UserWarning:
  warnings.warn(
```

✓ Model 2: With Address Feature

```
new_x = df.drop(["Price", "Price_USD"], axis=1)
new_y = df["Price_USD"]
```

```
msk = np.random.rand(len(df)) < 0.8
```

```
X_train = new_x[msk]
X_test  = new_x[~msk]
```

```
y_train = new_y[msk]
y_test  = new_y[~msk]
```

```
regr.fit(X_train, y_train)
print("Coefficients: ", regr.coef_)
print("Intercept: ", regr.intercept_)
```

```
Coefficients: [ 2.73544191e+03 -5.75249501e+04 -4.27078078e+04  5.92414896e+03
-2.09100434e+04  3.42748323e+03 -1.13525738e+06  2.20535058e+04
-3.98883104e+04  5.45812022e+04 -2.79235754e+04  9.01465409e+04
 5.62692450e+04  6.83379015e+04  3.57890387e+04 -1.07558711e+05
 3.90633720e+04 -6.39545870e+03  1.26049367e+05 -6.79506280e+04
-2.57114975e+05  1.25592439e+05 -7.49168165e+04  3.03803504e+04
 5.51567730e+04 -3.99087379e+04  3.18371564e+04  1.63656854e+04
 3.45607718e+04  1.01863407e-10 -3.50044932e+03  3.23661211e+04
-1.86998196e+05  5.96628524e-10 -2.93580767e+03 -6.08134489e+04
-8.13172187e+03  2.91038305e-11  3.70786324e+04 -6.91676098e+04
-4.08689299e+04  3.99396597e+04  2.69489395e+04  1.41621182e+05
-5.80533668e+03  2.73470371e+04  2.42081611e+03  9.69261298e+02
 3.04968055e+05  1.01787206e+04 -1.75683511e+04 -3.63797881e-10
 3.51594664e+04  5.56096951e+04  1.86804166e+05 -4.52182560e+04
-4.15156794e+03  1.04139077e+05  4.19923491e+04  9.35611449e+05
-3.52624657e+04  3.06851436e+04  9.70228878e+04 -4.11600917e+03
 1.11397422e+03 -1.49472076e+04  1.85158988e+03 -8.96524566e+04
-1.17023464e+04  3.89418650e+04  9.34898307e+04 -1.93943608e+04
-4.34920084e+04 -4.92912791e+04  3.92503738e+04 -1.24527432e+03
-9.53998409e+04 -1.45519152e-10  3.24873467e+04 -7.58029800e+04
-1.47715334e+04  1.85313860e+05  4.28696117e+04  3.60488772e+04
-6.02136469e+04 -3.76419702e+02  1.16415322e-10  8.79012112e+04
 4.29514498e+04 -1.06319259e+04 -7.43814978e+04  5.89557735e+05
-4.10279116e+04 -3.93636803e+04  1.41470802e+04 -7.83168533e+04
 1.31255427e+05 -3.62543765e+04 -3.02155431e+04 -1.50777264e+05
-3.06166480e+04  6.99801910e+04 -3.40428573e+04 -3.31057511e+04
-8.14619563e+03 -1.66471206e+03  4.16195755e+02  8.02801442e+04
 1.02820986e+05  1.90523398e+05  2.59040345e+04  3.53658664e+04]
```

```

-1.21040227e+04 -4.81244923e+04 -1.31449794e+04 1.43523045e+04
2.69251041e+04 -3.72080488e+04 -7.83973688e+04 9.40783559e+04
1.50354328e+04 3.40699182e+04 -1.57713049e+04 -2.26426268e+04
6.62669434e+03 7.01566981e+04 5.73002443e+03 1.49620242e+04
8.64264903e+04 2.73663748e+04 2.19859994e+04 -4.02975207e+04
8.58359246e+03 7.24921824e+04 1.03618209e+05 3.21294196e+04
-1.69194089e+04 1.36648326e+05 -7.97339423e+04 3.65629430e+04
-1.51609019e+05 4.65858966e+04 -1.02251500e+05 2.94582344e+03
-4.54560211e+04 3.92837892e+04 -4.84315230e+03 -1.11089556e+05
-1.15107923e-12 -1.20007524e+05 3.94909357e+03 -7.22051423e+04
-3.28117110e+04 1.55429177e+04 6.78331479e+04 6.55721222e+02
1.68733346e+04 -1.33857284e+04 -2.03540656e+03 5.61465612e+03
-2.39764812e+02 4.89930764e+03 -2.84990313e+04 8.88055868e+04
4.03196151e+04 -1.50412337e+04 -7.49896010e+04 1.01420158e+04
6.17038664e+03 -2.58948518e+04 -7.47152343e+04 -2.58060164e+04
4.06316668e+03 -2.86504119e+04 -2.92123985e+04 5.00874575e+04
1.11197657e+04 6.94143642e+04 5.53882443e+04 3.65025034e+04
-3.72247168e+03 5.41304809e+05 8.25239113e-08 2.22366109e+05
-6.82121026e-11 9.75999938e+03 5.95614996e+04 2.65160297e+03
-1.73327668e+04 3.10879585e+04 3.34831932e+04 3.11855962e+03
-3.12996523e+04 2.70473964e+05 -3.14251782e+04 -7.26754950e+04
1.00264117e+05]

```

Intercept: -1115744.5215320422

```
from sklearn.metrics import r2_score
```

```

y_hat = regr.predict(X_test)
print("MSE: %.2f" % np.mean((y_hat - y_test) ** 2))
print("R2-score: %.2f" % r2_score(y_test, y_hat))

```

MSE: 12081987826.53

R2-score: 0.76